1D COMPRESSIBLE FLOW

Mech 588 – Winter 2023 UBC Mechanical Engineering

Christian Rowsell
40131393

Solve the following shock tube problem to t = 0.15: (r_L = 6 u_L = 0 P_L = 12), (r_R = 1 u_R = 0 P_R = 1); set your domain size appropriately to prevent any waves from reaching the boundary before that time. Use at least two of the Steger-Warming, Roe, and AUSM+ schemes[1], and a mesh with at least 200 points. You should use a second-order accurate discretization in both time and space; this implies that you will need to use a limiter of some sort to preserve monotonicity. Your writeup should identify which schemes you used (including limiter, time-stepping scheme, and time step). Submit plots of r, u, P, and T (one plot for each variable, but all schemes plotted together).

The following report discusses the solving of the compressible Euler equations, with the goal of solving the shock tube problem. The final code for this project has functionality for 3 different flux calculation schemes, as well as 3 different time marching schemes. These are as follows:

Time Schemes	Flux Schemes
Forward Euler (1st Order)	Roe's Flux Difference Splitting
Runge-Kutta 2 (2nd Order)	Steger-Warming Flux Vector Splitting
Midpoint (2nd Order)	Lax-Friedrich

Unfortunately, I was unable to figure out how to implement the limiter. The functionality is already coded for the most part, but it is not implemented. As such all simulations that are demonstrated will have overshoots near discontinuities. This report will discuss code functionality, how to set up the code to run simulations, code testing, and finally will give the results for the shock tube problem. A make file is included in the submission package. This make file has the following functionality:

- make run: Running the make run command will compile any code required to create the run executable. It will also create the test executable, and run all the tests. In the event that a test fails, the user will be prompted to which test failed, and in what way did it fail.
- make all: Running the make all command will compile all of the code required to create the run and test executables. This function differs from the make run command, in that the make run command will not recompile code that has not been updated, while the make all command will.
- make clean: The make clean command will remove all object files and executables.
- make clearresults: The make clearresults command will remove all files from the results folders.

To increase ease of use, a input.txt file is included. The code will read all results from this input.txt file. To run the code, simply compile the code, and run the ./run command. If the user specifies an input file (such as the input.txt file), by running ./run *FILENAME*, the code will run the simulation with the desired parameters specified in the file. If a file is not provided, the code will run using the following default parameters:

// Assign the default values to provide a contact discontinuity

CFL = 0.95;

nx = 200; // number of cells

xbegin = 0;

xend = 1;

```
gamma = 1.4;
init_cond[0] = 1.01; // left density
init_cond[1] = 0; // left velocity
init_cond[2] = 1.0; // left pressure
init_cond[3] = 1.0; // right density
init_cond[4] = 0; // right velocity
init_cond[5] = 1.0; // right pressure
t_final = 0.15;
num_save = 20; // How often to save results
timescheme = 2; // Midpoint default
limitscheme = 0; // superbee default
```

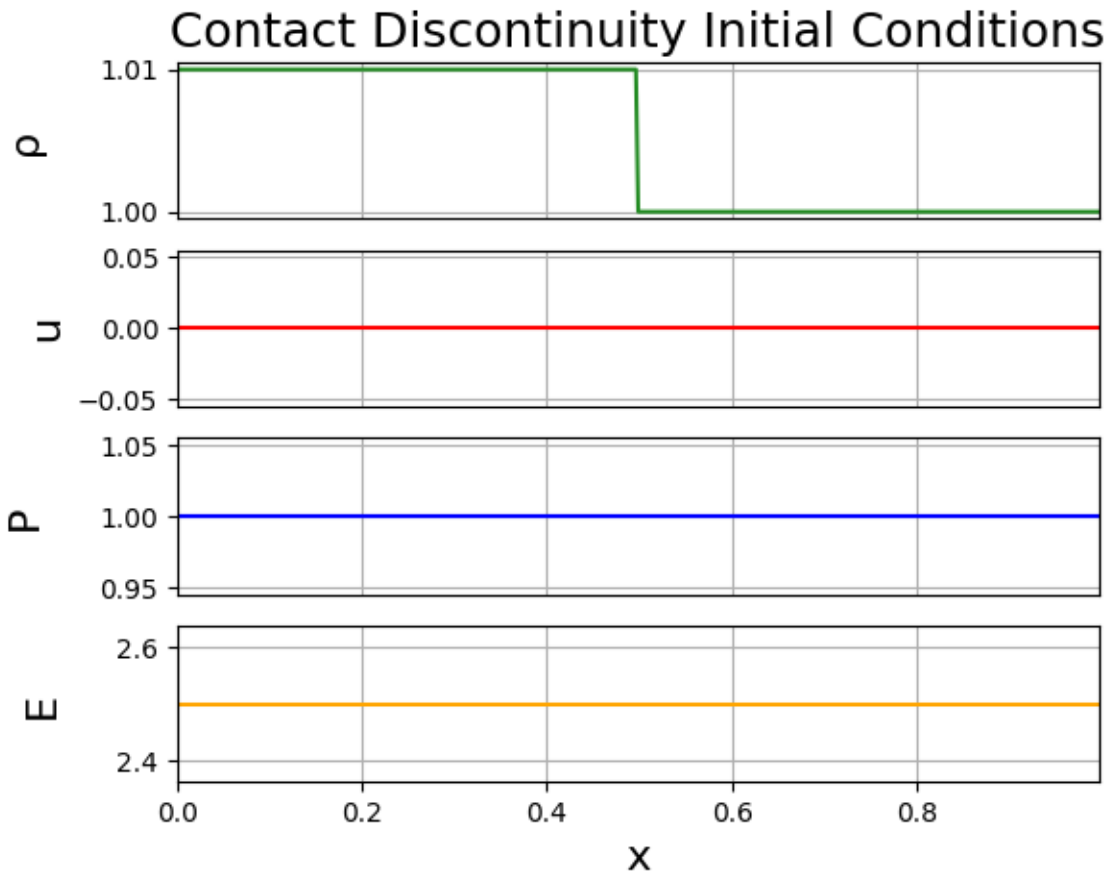
It can be noticed that the code allows the user to select a time scheme, as well as a limiter scheme. As previously stated, the limiter is not implemented, and as such this line can be left alone. The timescheme can be changed to forward Euler using 0, RK2 using 1, and midpoint using 2. The simulation will run for all 3 flux schemes, saving all files in their respective folders. The initial and final conditions will be saved in the Results folder, while data at individual timesteps will be saved in their respective folders, RoeTimesteps, StegerTimesteps, and LaxTimesteps. A python script is also included to allow for plotting of this data. The Plot_Single function can be used to plot a single timestep, or result, the Plot_Timesteps function can be used to plot the timesteps of a run sequentially and the Plot_Results function can be used to plot results from all 3 simulations on the same graph.

A variety of testing functions were created to ensure that everything works. These won't be discussed in detail, but they include functions to test matrix multiplication, matrix assembly, Jacobian calculation, computation of maximum stable timestep, and flux integral calculation.

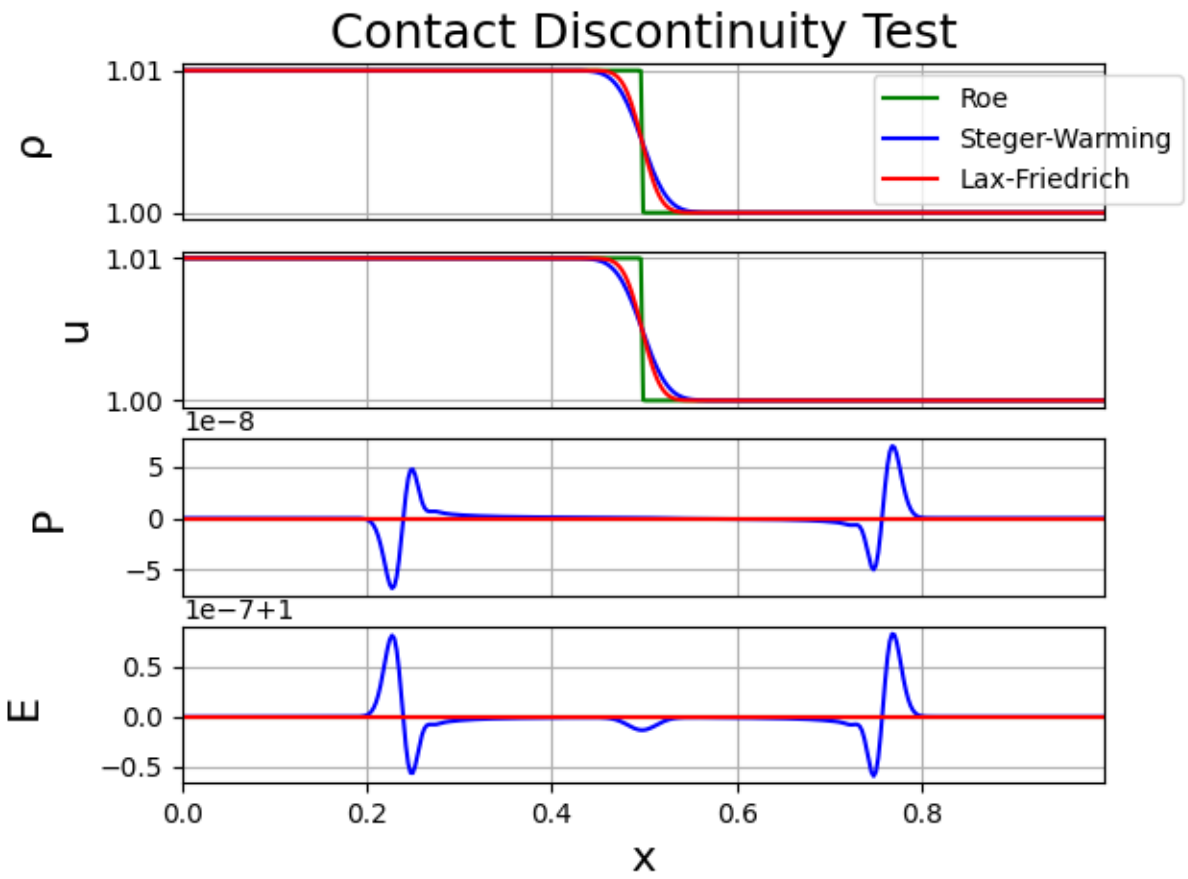
The following is the results for the stationary contact discontinuity test case. These simulations were ran with the following parameters:
CFL: 0.9
nx: 401
xbegin: 0
xend: 1
gamma: 1.4
rhol: 1.01
ul: 0
Pl: 1.0
rhor: 1.0
ur: 0
Pr: 1.0
tfinal: 0.15

num_save: 10
timescheme: 2
limitscheme: 0

The following shows the initial conditions given these parameters



The following shows the solution, at time t=0.15 for each scheme.



The next text case involves the weak pressure wave. These results were obtained using the following run parameters:

CFL: 0.6

nx: 401

xbegin: 0

xend: 1

gamma: 1.4

rhol: 1.0

ul: 0

Pl: 1.01

rhor: 1.0

ur: 0

Pr: 1.0

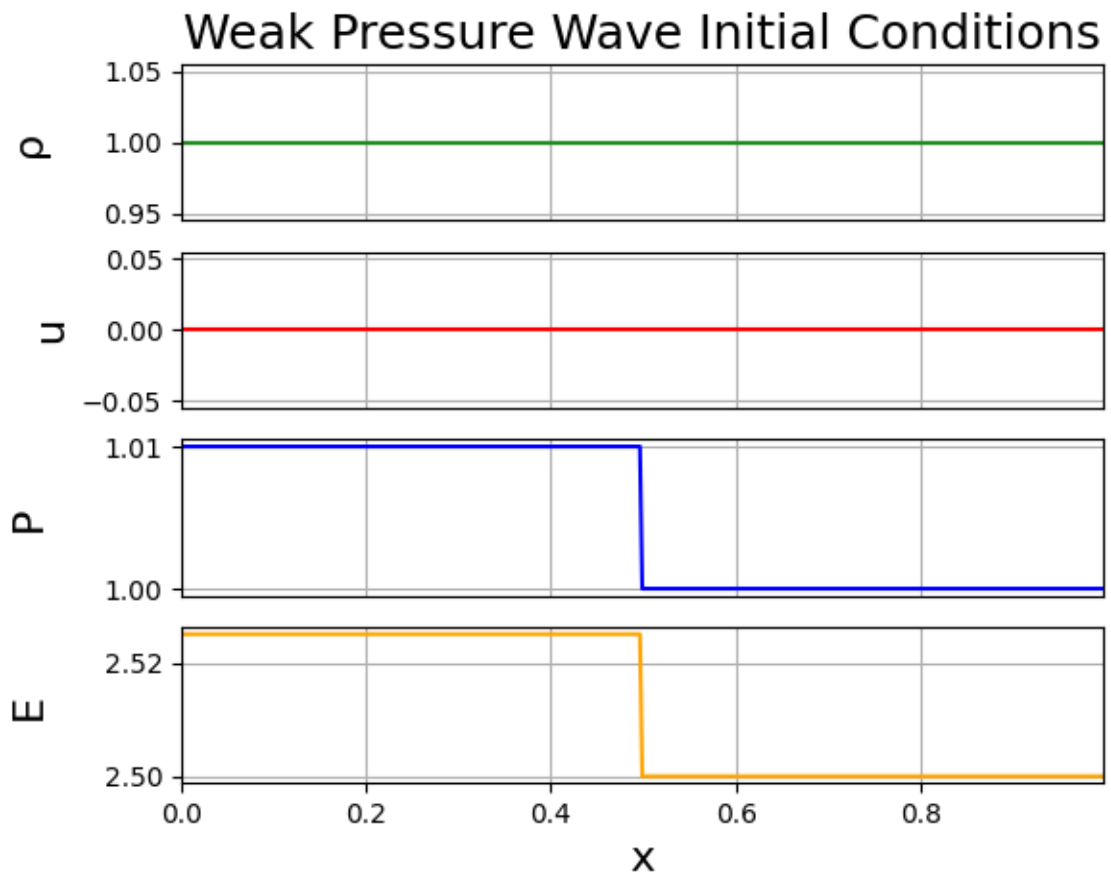
tfinal: 0.15

num_save: 10

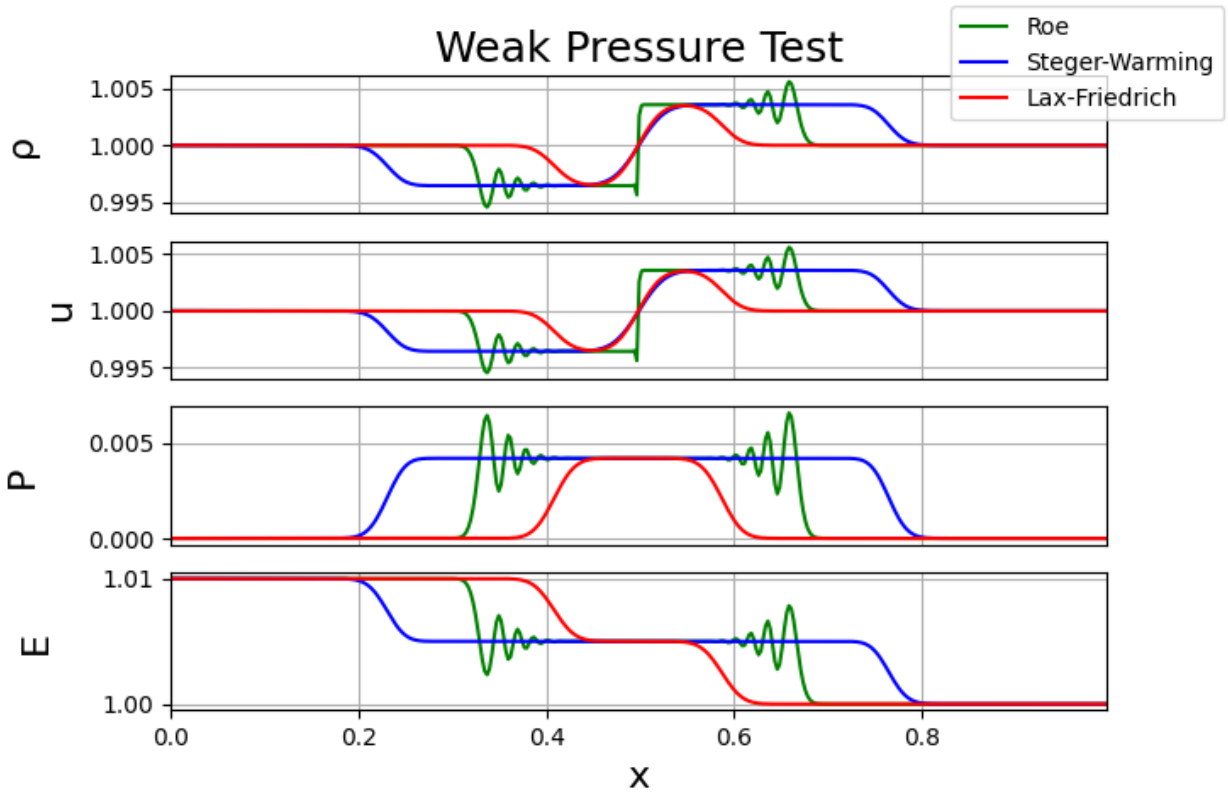
timescheme: 2

limitscheme: 0

The following graph shows the initial conditions given these test parameters.



The following graph shows the results for this test, at a time of t=0.15.



Finally, the strong shock test case is performed. The following parameters were used for this test case.

CFL: 0.05

nx: 801

xbegin: 0

xend: 2

gamma: 1.4

rhol: 6.0

ul: 0

Pl: 12.0

rhor: 1.0

ur: 0

Pr: 1.0

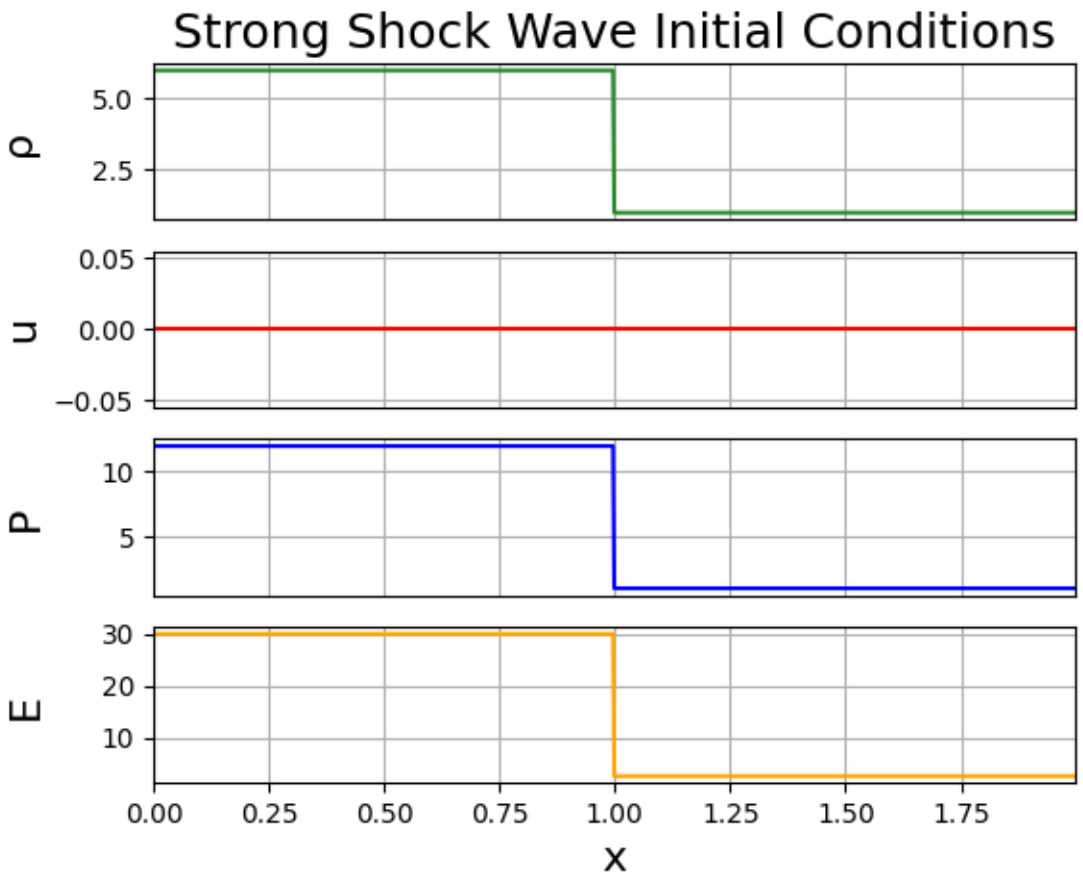
tfinal: 0.15

num_save: 10

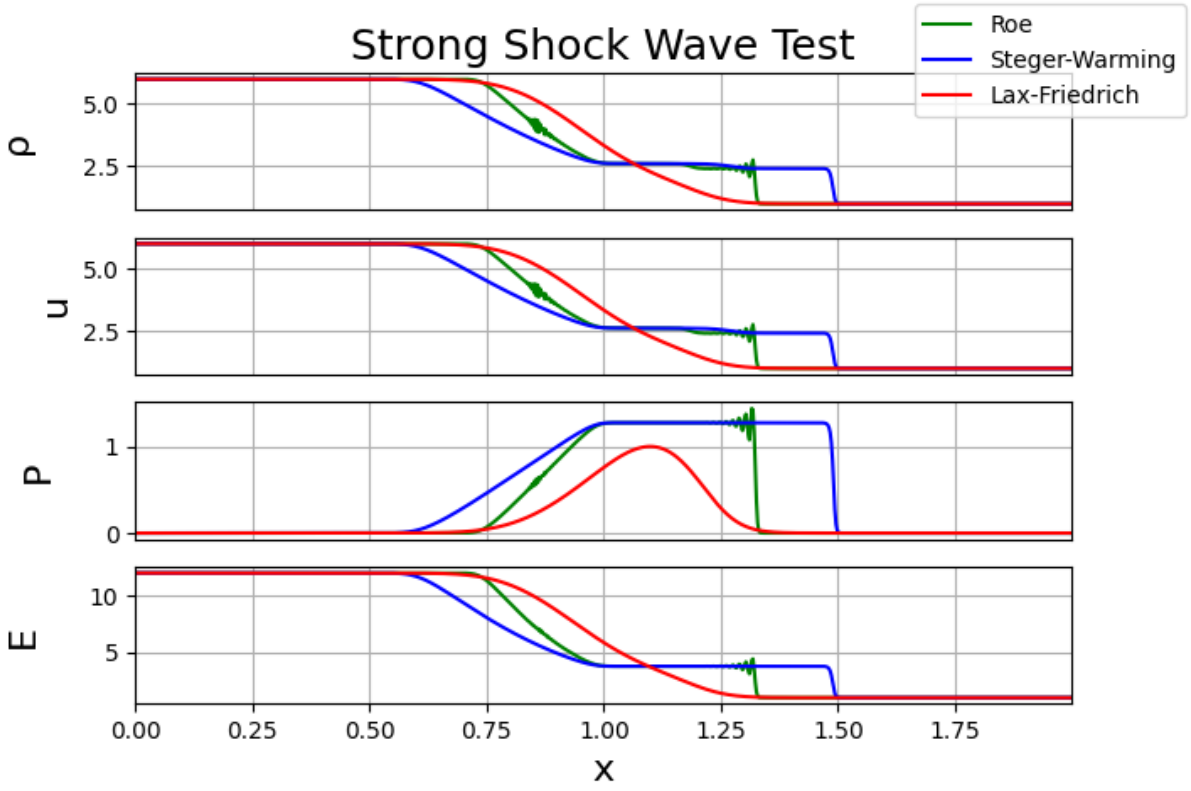
timescheme: 2

limitscheme: 0

The following graph demonstrates the initial conditions for this test.



Finally, we can see the results for this test.



Overall, the code works as expected. The results match those given in the class notes. It can be noticed that the CFL number must be dropped as the test cases get harder. This is because the Roe flux scheme begins to oscillate very aggressively at a higher CFL number, becoming unstable in nature. Without a flux or slope limiter this is to be expected. In the future, a limiter could be implemented, allowing these overshoots to be smoothed out, and allowing the solution to become stable, even at higher CFL numbers. Having to put such a low CFL number makes the simulations very expensive computationally. Given the same domain, and grid spacing, the code is speed up 45x, coming in at around 18 seconds runtime with a CFL number of 0.05 and 801 grid points, while only having a runtime of about 0.4 seconds with a CFL number of 0.95. On larger, more complex problems, these time savings are essential, and as such a limiter must be implemented to use this code further. The results for each of the flux schemes is as expected. The Roe flux scheme gives the sharpest results, while also facing the worst oscillations. The Steger-Warming scheme is more diffusive, often smoothing the results. Finally, the Lax-Friedrich scheme (which was not assigned to us, but I wrote as a higher order flux scheme while trying to figure out how to implement flux limiting), does not give very good results at all. This is expected, as this scheme does not implement any flux splitting, or upwinding.